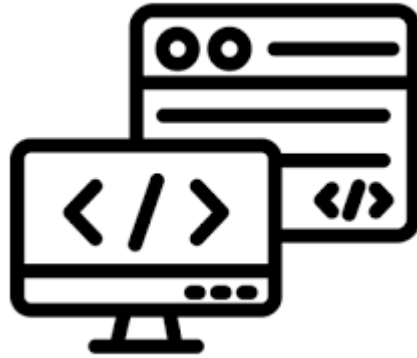




Stage III:

Backend

Autómatas, teoría de lenguajes y compiladores 72.39

Grupo:

Federico Etchegorry 63175
Mariano Iván Odzomek 63386
Tomás Raiti 62887
Thomas Ruijgt 62875

Tabla de contenidos:

1. Introducción	3
2. Modelos computacional	3
2.1 Dominio:	3
2.2 Lenguaje:	3
3. Implementación	5
3.1 Frontend:	5
3.2 Backend:	5
3.3 Dificultades encontradas:	6
4. Futuras extensiones	6
5. Conclusiones	7
6. Anexo	7
7. Bibliografía	7

1. Introducción

El presente trabajo es un sistema de domain specific language (DSL) diseñado con el propósito de simplificar la comunicación de ideas en lo relacionado a estructuras de información. El mismo hace que sea sencillo desarrollar un diseño y fácilmente cambiar de tipo de estructura según se va discutiendo en el grupo. También permite la customización de los nodos para mejorar la interpretación de la estructura y desarrollar conceptos más complejos.

2. Modelos computacional

2.1 Dominio:

El lenguaje es una combinación de JSON para el orden de las estructuras, con modificadores de tipo anotación para las customizaciones, las cuales se escriben en un formato de property(rule) y permite el uso de variables estilo bash.

El resultado del mismo se genera en formato dot (GraphViz), el cual es sencillo de visualizar, ya sea con extensiones de VS Code o en un playground online, y permite una vasta cantidad de opciones de customizaciones. El DSL se encarga de abstraer esta capa de lógica para que sea más sencillo de utilizar y graficar estructuras típicas de programación.

2.2 Lenguaje:

Al inicio de una estructura es posible declarar variables de la forma:
`$variable: color(blue) border(dotted) ...`

Luego, se declaran las customizaciones que se quieren definir para los diferentes tags con `@Customize(TAG, color(red) border(solid))` y se pueden usar variables dentro de los atributos `@Customize(TAG, $variable)`. En caso de definir múltiples customizaciones se tomará el último valor que aparece para cada uno de los atributos.

Posteriormente, es posible listar una serie de atributos default (el lenguaje tiene atributos por defecto en caso de que el programa no incluya uno) de la misma forma que los customize, sin tener que declarar etiquetas: `@Default(color(red) border(solid))`, también soportan el uso de variables dentro de estos. Es importante remarcar que las variables son globales posterior a su declaración, no así los customize y el default que deben declararse justo antes de cada estructura que necesita usarlos.

Una vez finalizada la etapa de customización se definen 1 o más estructuras. Un programa de nuestro lenguaje permite definir las siguientes estructuras (todas son válidas con {...} o con [...], empleando {} si es requerido que los elementos de la estructura estén ordenados):

- Array ["Elemento 1", ...] ;

- List ["Elemento 1", ...]; o LinkedList["Elemento1", ...]; o DoubleLinkedList ["Elemento 1", ...];
- Tree["Elemento 1", ...,
 { "Elemento n", "Elemento m", ... }
];
 donde cada al abrir '{' o '[' permite crear un nodo con hijos que salen del primer string de la subcelda.
- Graph[A: {"Elemento 1", B, ...},
 B: {"Elemento 2"}
];

Cada etiqueta listada dentro de las llaves representa una arista entre el nodo afuera de las llaves y el nodo dentro de ellas. El string del nodo debe aparecer antes de los nodos apuntados y si se usan varios string solo se tomará el primero que aparece.

- DirectedGraph que emplea la misma estructura que Graph.
- Table[{"Fila1Columna1", "Fila1Columna2", ...},
 {"Fila2Columna1", "Fila2Columna2", ...},
 ...];

Donde las llaves se usan para indicar las filas de las tablas y se puede definir una cantidad distinta de columnas para cada fila, las cuales quedarán vacías en la tabla final.

Nuestro lenguaje no soporta anidar estructuras. Si se desean hacer comentarios de línea es válido el uso de # y //. Para hacer un bloque de comentario se puede usar /* ... */.

Ejemplo de un programa con todo lo explicado anteriormente:

```
//esto es un comentario
$variable: border(solid);
@Customize(A, $variable)
@Customize(B, border(dotted))
@Default(color(red) fontSize(25))
Array:[
  A:"primero",
  "otro elemento"
];
@Customize(A, $variable)
@Customize(B, border(dotted))
@Default(color(green) fontSize(25))
Graph:{
  A: {
    "La letra A",
    B,
    C: { "Content", A, B, C }
```

```

    },
    B: {"La letra B", C, E },
    D: "Alone",
    E: { "Algo", "Algo más", "Otra cosa más" }
  };
  @Default(color(blue) fontSize(30))
  Tree:[
    "tercero",
    "segundo del tercero",
    "tercero del tercero"
  ];

```

3. Implementación

3.1 Frontend:

El front end se desarrolló teóricamente diseñando las estructuras del AST de forma visual, las cuales luego se tradujeron a structs de C.

Luego, se establecieron los tokens que se utilizarían en el lenguaje. Estos regexs sufrieron grandes cambios durante toda la etapa, hasta que finalmente se llegó al diseño final. Para la parte de Bison, se trabajó mediante pruebas, warnings y errores, hasta que se obtuvo la solución final, la cual forma el árbol AST utilizando los structs definidos en el primer paso.

Para evitar correr el código de backend durante esta etapa del desarrollo, se utilizaron las poco-hombre gotos, a modo de saltar estos pasos del código.

3.2 Backend:

La etapa de backend se dividió en dos subetapas. La de validación semántica y la etapa de generación del archivo.

En la etapa de validación, se valida cada estructura que se encuentra dentro del struct Structure generado por Bison. Por cada estructura se comienza por analizar y validar las variables de estilo, siguiendo por las anotaciones, las etiquetas, y finalmente la estructura en sí. De fallar en alguno de estos pasos, se aborta inmediatamente.

En caso de que la validación haya sido exitosa, se continúa a la etapa de generación del programa de salida. En la misma se delega el armado del programa de salida en distintas funciones, que corresponden a las distintas estructuras que son soportadas. La salida generada es uno o varios archivos .dot, que se podrán encontrar en la carpeta output en la raíz del proyecto. Se puede optar por no generar esta carpeta al

utilizar la flag `--dry-run`, lo que imprimirá el output en la salida estándar. La misma es utilizada para poder correr los test sin generar ninguna carpeta adicional.

3.3 Dificultades encontradas:

El desarrollo del proyecto resultó en general bastante fluido, aunque enfrentamos algunos inconvenientes que logramos resolver con rapidez.

Al comenzar, nos dimos cuenta de que no sería posible aplicar múltiples estructuras al mismo tiempo. Para abordar este desafío, optamos por generar un archivo separado para cada output, simplificando el proceso.

Más adelante, cuando el backend aún estaba en desarrollo y el frontend ya estaba completo, surgieron errores inesperados en pruebas que habían funcionado correctamente durante la segunda entrega. Estos problemas se manifestaron al manejar cadenas vacías y al intentar incluir saltos de línea dentro de un string. Para solucionarlo, decidimos no permitir este tipo de entradas. En el caso de cadenas vacías, se estableció que debían representarse mediante un string que contenga un único espacio. Mientras que para los saltos de línea optamos que sea rechazado a nivel léxico.

4. Futuras extensiones

De cara a futuras extensiones, el proyecto presenta un amplio potencial de crecimiento y mejora. Entre las principales posibilidades se encuentra la incorporación de soporte para estructuras anidadas, lo que permitiría abordar casos de uso más complejos y aumentar la flexibilidad del sistema. Un ejemplo de esto sería la implementación de tablas que apunten a listas, logrando una funcionalidad similar a un hash-map.

Otra línea de desarrollo relevante es la personalización estética de los resultados, incluyendo la adición de estilos visuales que faciliten la interpretación y presentación de los datos. Esto podría abarcar opciones para configurar formatos y tipografías, adaptándose a diferentes necesidades y preferencias.

Adicionalmente, se plantea ampliar la capacidad de procesamiento para manejar un rango más amplio de variables y formatos. Esto permitiría trabajar con datos más complejos y provenientes de diversas fuentes, incrementando la robustez y compatibilidad del sistema.

5. Conclusiones

En resumen, este trabajo práctico permitió explorar la implementación de un analizador léxico y profundizar en las herramientas necesarias para su desarrollo, como Flex y Bison. A lo largo del proceso, se trabajó en comprender la teoría detrás de la construcción de autómatas y en aplicarla de manera práctica mediante el desarrollo realizado.

Si bien surgieron desafíos, como el planteamiento de la gramática y la correcta definición de estados en el analizador, estos fueron resueltos mediante iteración y mejoras en el enfoque. Esto demuestra que los errores forman parte del aprendizaje y que el desarrollo de software no siempre sigue un camino lineal.

En conclusión, este trabajo práctico aportó herramientas valiosas y un mejor entendimiento de las tecnologías relacionadas con la teoría de lenguajes formales. Además, permitió implementar un primer compilador funcional, lo que representa una experiencia enriquecedora tanto a nivel técnico como en el desarrollo de habilidades analíticas.

6. Anexo

<https://magjac.com/graphviz-visual-editor/> sitio web para ejecutar los output de nuestro programa.

<https://raw.githubusercontent.com/Secreto31126/TP-TLA/refs/heads/development/doc/asset/ast-tree.svg> ejemplo visual de AST del lenguaje, generado con el TP presentado.

7. Bibliografía

No se utilizaron recursos más allá de los provistos por la cátedra.